



Bassir — Owner Eyes

Developer Handoff — Remaining Work & Go-Live Guide

Repository: `invetechsl/BEVIP-OWNER-EYES` · Branch: `claude/bassir-owner-eyes-system-aogee5`

Prepared for the development team · September 2026

1. Purpose of this document

The application is **feature-complete and deployable**. This document tells your developer **what is left to do to take it live and beyond the demo** — mostly provisioning, configuration, connecting real external services, and a few enhancements. It complements the two existing technical docs in the repo: `docs/DEVELOPER-HANDOFF.md` and `docs/WHAT-WAS-ADDED-EN.md`, and the deployment guides `DEPLOY.md` / `DEPLOYMENT.md`.

Priority key

P0 Go-live essentials (do before launch) · **P1** Real integrations & hardware · **P2** Enhancements & polish

2. What is already built & working

So the developer knows the baseline (no work needed here — verified, tests green):

- **Full 5-role system** (owner, owner rep, consultant, contractor, admin) with per-role pages and data scoping.
- **Real backend**: Node HTTP server (no framework), **SQLite** persistence (transactional, WAL) via `node:sqlite` — JSON fallback available. File uploads persisted to disk.
- **Authentication** with signed tokens (`JWT_SECRET`), role-based authorization.
- **Real service integrations**, each enabled by an env var and otherwise running in clearly-labelled simulation mode: Email (Resend/SendGrid), WhatsApp (Meta Cloud API), AI image analysis (Anthropic), camera snapshot ingress, RTSP live streaming (MediaMTX).
- **PDF reports** generated server-side (pdftkit + Arabic reshaping).
- **In-browser viewers, fully offline**: 3D BIM/IFC, vector DXF, and PDF drawings — no external/paid services.
- **Contractor BOQ & Schedule** pages with file parsing (CSV/Excel/Primavera XER/P6 XML/PDF), baseline lock, and dual-approval revisions.
- **Progress Map** that renders your **real uploaded drawings** and brightens dark → bright per zone as work is completed and approved.

- **Saudi Arabia project map** (newest feature — see §6).
- **Automated regression tests** (Playwright) — smoke, full crawl (0 JS errors, all roles), brightness, end-to-end cycle, BIM render.

3. P0 Go-live essentials (configuration, not code)

Item	What you need to do	Where
Server, domain & HTTPS	Provision a host, point a domain, and put the app behind a reverse proxy with TLS. Docker is the recommended path.	<code>DEPLOY.md</code> §3–§4, <code>Dockerfile</code> , <code>docker-compose.yml</code>
Production secret	Set a long random <code>JWT_SECRET</code> . If unset it is auto-generated to <code>data/.jwt-secret</code> (fine for one host, but set it explicitly for production/redeploys).	<code>.env</code> (see <code>.env.example</code>)
Real vs demo data	Important: <code>npm run seed --reset</code> loads demo data. For production, start clean and create real projects/users through the UI. Change every default password.	<code>server/seed.js</code> , Users page
Set <code>APP_URL</code>	The public URL used inside automatic email/WhatsApp notification links.	<code>.env</code>
Backups	Schedule backups of <code>data/bassir.db</code> (SQLite) and the uploads folder.	<code>DEPLOY.md</code> §6
Pre-flight & tests	Run <code>npm run preflight</code> and <code>npm test</code> before launch; both should pass.	<code>tools/preflight.js</code> , <code>tests/</code>

4. P1 Real integrations & hardware

Each of these works in simulation until you provide credentials/hardware. Enable the ones the client wants live.

Item	What you need to do	Where
Email notifications	Add <code>RESEND_API_KEY</code> (or <code>SENDGRID_API_KEY</code>) + <code>EMAIL_FROM</code> to send real report emails and review-cycle notifications.	<code>server/integrations.js</code> , <code>.env</code>
WhatsApp	Add <code>WHATSAPP_TOKEN</code> + <code>WHATSAPP_PHONE_ID</code> (Meta WhatsApp Cloud API) for real WhatsApp delivery.	<code>.env</code>
Live cameras	Deploy MediaMTX and set <code>MEDIA_SERVER_URL</code> for RTSP → HLS/WebRTC live view; set <code>CAMERA_KEY</code> and point the NVR/cameras to POST snapshots. Camera URLs in seed data are placeholders.	<code>DEPLOY.md</code> §5, <code>.env</code>
AI visual monitoring	Add <code>ANTHROPIC_API_KEY</code> to enable real site-photo analysis. For fully automatic per-element progress % detection, scope a dedicated CV pipeline	<code>.env</code> , <code>server/integrations.js</code>

	with stakeholders — insights beyond keyed analysis are simulated seed data today.	
Project drawings (2D)	Now working — upload a real drawing per floor/zone and it appears on the Progress Map, lighting up executed zones as the BOQ is approved (see §6). Remaining: upload the full drawing set for every floor/zone of each live project.	<code>public/js/views2.js</code> , <code>public/js/views.js</code>
BIM/IFC per project	Replace the sample tower IFC with each project's real IFC. Floor-level lighting from BOQ already works; per-element lighting remains (see the BIM row in §5).	<code>public/vendor/bim/</code> , <code>public/js/bim-viewer.js</code>
Schedule/BOQ parsing at scale	Validate the parsers against your real Primavera/Excel exports; consider moving very large-file parsing server-side.	<code>public/js/views5.js</code>
Coordinates backfill	New projects now require lat/lng. Add coordinates to any pre-existing/imported project via Project Settings → "إحداثيات موقع المشروع" or it will not appear on the map (see §7).	Project Settings page

5. P2 Enhancements & polish

Item	What you need to do
Finish English i18n	Some UI strings still fall back to Arabic when English is selected. Fill missing translation keys in the <code>I18n.registerDict</code> blocks.
CI/CD	Add GitHub Actions to run <code>npm run preflight</code> + <code>npm test</code> on every PR.
Server integration tests	Current Playwright tests run in browser demo mode. Add tests that hit the real API + SQLite.
Native mobile app	A design handoff exists in <code>docs/mobile-design-handoff/</code> ; the native app itself is not built.
Security hardening	Rate limiting, secure/HTTP-only cookies, CSP headers, audit-log retention/rotation, and routine dependency updates.
BIM/IFC per-element linking	The 2D Progress Map now renders the real uploaded drawing and lights zones from approved BOQ (done); the 3D BIM tab lights up per floor from BOQ (done). What remains is per-element linking: map each IFC element (by GUID) to its BOQ item so individual components light up. This needs an IFC parse + a mapping step (UI or import), then the viewer colors elements by their BOQ progress.

6. Progress Map now runs on your real drawings

Previously the Progress Map drew a synthetic schematic, so uploaded drawings never appeared and it "looked like a demo." Fixed:

- Upload a drawing from **BIM Models & Drawings** → "رفع مخطط/نموذج", tagged by **floor**, **zone** (whole floor or one of 6 zones), and discipline.
- The Progress Map **2D Drawings** tab renders that **actual uploaded drawing** as the (dimmed) background; the executed zones **light up over the real plan** as their BOQ is approved. A clear call-to-action shows for floors with no drawing yet.
- Each zone can open its own uploaded drawing; the side panel lists the linked BOQ items.
- **Demo note:** uploads now persist the file content (data URL in demo, disk on the server) so drawings are actually viewable — this is why the page is no longer "just a demo."

Code: `public/js/views.js` (`render2d` , `floorPlanDrawing` , background render), `public/js/viewer.js` (`renderPlanInto`), upload + zone field in `public/js/views2.js` , demo file persistence in `public/js/api.js` .

7. Earlier feature — Saudi Arabia project map

Added to the **owner dashboard** (bottom) and the **owner-rep portfolio** page:

- A real, **offline** map of the Kingdom (projected boundary baked into the code — no external map tiles).
- One dot per project with traffic-light status:
 - **Green** — on time & on budget
 - **Yellow** — needs action (before red)
 - **Red** — behind schedule or over budget

- **Click a dot** → **opens that project** (switches the active project and its dashboard). Hover shows name, status, progress %, days behind, and budget overrun.
- **Coordinates are required when the owner rep adds a project** (validated to be inside the Kingdom), with a 29-city quick-picker that auto-fills lat/lng. Existing projects can be given coordinates in **Project Settings**.

Code: map rendering + status logic in `public/js/views.js` (`projectsMapHtml` , `mapStatus` , baked `KSA_PATH`); add-project form in `public/js/views2.js` ; edit/backfill in `public/js/views4.js` ; `lat` / `lng` persisted in `shared/api-core.js` .

7. How to run & test

Run locally

`npm install` → `npm start` → open `http://localhost:3000`

Docker (recommended for the server) — see `DEPLOY.md` §3a.

Tests

`npm run preflight` — static checks

`npm test` — smoke + full crawl + brightness + cycle + BIM

Individual: `npm run test:crawl` , `test:brightness` , `test:cycle` , `test:bim`

8. Appendix — key files & demo accounts

Where things live

- `public/js/app.js` — routing, navigation, page registry, project switcher.
- `public/js/views.js` ... `views5.js` — page renderers (dashboard, portfolio, BOQ, schedule, etc.).
- `shared/api-core.js` — business logic engine (runs in both server and browser demo).
- `shared/seed-data.js` — seed/demo data + schema version.
- `server/` — `server.js` , `storage.js` (SQLite), `integrations.js` , `pdf.js` , `reports.js` , `seed.js` .
- `DEPLOY.md` / `DEPLOYMENT.md` — deployment; `.env.example` — all config keys.

Demo accounts (change for production)

Role	Username	Password
Owner	<code>owner</code>	<code>owner123</code>
Owner rep	<code>rep</code>	<code>rep123</code>
Consultant	<code>consultant</code>	<code>consult123</code>
Contractor	<code>cont-arch</code>	<code>cont123</code>
Admin	<code>admin</code>	<code>admin123</code>